Claude Desktop 實作

Vibe Coding
跨領域實作工作坊

不用寫程式，也能把你的點子做成能跑的作品

醫療　人文　商管　法律

GET READY

先做三件事

1. **下載 Claude Desktop**
   前往 https://claude.com/download 先下載

2. **收 Team 邀請信**
   我會把你的 Gmail 加入學校的 Team，收到信後按「接受」

提示｜帳號加入需要一點時間，下載完先安裝，等一下再登入。

SETUP · DESKTOP

安裝並登入 Claude Desktop

1. **下載並安裝**
   https://claude.com/download ，雙擊安裝，全部保持預設值

2. **登入**
   接受 Team 邀請後，用同一個 Gmail 登入；會跳瀏覽器，再自動回到 App

3. **設定語言**
   點左下角你的帳號 → Language，選「中文（繁體）」

4. **打開 Chat 分頁**
   第 1 章先用 Chat 分頁。Code 分頁等第 2 章裝好 Git 再開

> **提示**｜官方安裝說明：https://code.claude.com/docs/en/desktop

OVERVIEW

今天的流程

內容	你會做出什麼
AI 使用現況、怎麼寫好 prompt	會寫 prompt
第 1 章　Desktop Chat：文字分析與數據分析	文件摘要、營運分析儀表板
第 2 章　環境安裝、貪食蛇與語音網頁	貪食蛇遊戲、語音轉文字網頁，並部署上線
第 3 章　動態網站（RAG 聊天機器人）	根據你的資料回答的機器人
第 4 章　Skill	把好用的方法存成自己的 Skill
第 5 章　MCP（Playwright）	讓 Claude 自己測試網站並截圖
收尾與 Q&A	帶走的三件事

CONTENTS

目錄

章節	頁
開場、安裝 Desktop	2
第 1 章 Desktop Chat	19
1-1 文字分析（PDF、表單、發票）	21
1-2 數據分析（山嵐咖啡）	34
第 2 章 貪食蛇與語音網站	44
Plan Mode 與 prompt	51
語音轉文字網頁	63

章節	頁
第 3 章 動態網站：RAG 聊天機器人	72
進階功能（個性、歷史、語音）	84
第 4 章 Skill	92
存成 Skill 與測試	99
第 5 章 MCP	111
語音包成桌面 App	122
收尾	127

WARM-UP

這些 AI，你看過幾個？









提示｜不用說名字，看到眼熟的就舉手。哪些你**用過**？哪些只是聽過？

WARM-UP

這些 AI 的名字

ChatGPT
OpenAI

Claude
Anthropic

Gemini
Google

Copilot
Microsoft

Perplexity
搜尋型 AI

NotebookLM
Google，讀文件

Grok
xAI

你用 AI 做過什麼？

寫報告、查資料、翻譯、做簡報、寫程式⋯⋯ 說一個你覺得最有用、或最失望的經驗

BEFORE WE START

用 AI 常見的困境

你可能遇過	背後的概念	怎麼辦
答非所問	**提示詞 Prompt**：說得愈清楚，結果愈好	用 PTCF 寫清楚
一本正經說錯話	**幻覺**：很有自信，但內容是編的	一定要驗收
聊著聊著忘了	**上下文 Context**：一次能記住的量有限，對話太長就會忘	換題目開新對話
不敢給資料	資料會被送去處理，各單位規定不同	去識別化、依規定

提示｜AI（生成式 AI／LLM）像反應很快、知識很廣的實習生：很能幹，但需要說清楚，也需要檢查。

CHECK

確認帳號：四項都通過再往下

1. **左下角點你的帳號**
   顯示「東吳大學」，不是個人帳號（可從這裡切換）
2. **看上方的分頁：有 Chat 和 Code 兩個**
3. **在 Chat 傳一句「你好」，有收到回覆**
4. **把模型換成 Sonnet 5.5**
   點輸入框右下角的模型名稱，選 Sonnet 5.5。每個新對話都要看一次

> **注意**｜跳出「您已加入 東吳大學！」：選**保留兩個帳戶** → 繼續。**不要選「僅使用東吳大學」**，會關掉你的個人帳號。

BEFORE WE START

開始之前，先說清楚幾件事

- 今天用的資料都是**假資料**，請放心練習
- AI 會錯：數字、事實、連結，**一定要自己驗收**
- 真實的個資、病歷、合約、API key，**不要貼給 AI，也不要截圖分享**
- 作品能不能用在報告或作業，請依照你所在單位的規定

HOW IT WORKS

怎麼上這堂課

- 每個人用自己的電腦，自己動手做一遍
- 卡住就舉手；旁邊同學做得比較快，直接問也完全 OK
- 每題先把**基本版**做出來就很好，進階題做不完很正常
- 你會發現等 AI 的時間比自己寫還長，那就趁空檔想下一個要求要怎麼講

提示｜今天不會有人檢查程式碼寫得好不好，只看成品能不能動、是不是你要的。

CONCEPT

Vibe Coding 是什麼

你的工作	AI 的工作	不變的事
描述目標與規則	拆解、規劃、寫檔案	錯的成品還是錯的
看成品、挑毛病	執行、除錯、重試	你看不懂就不能收
決定要不要收	解釋它做了什麼	責任仍然在你身上

小結｜今天要練的肌肉：把「我想要一個很酷的網站」，翻譯成「有哪些頁面、按下去會發生什麼事、什麼情況算做壞了」。

CONCEPT

Chat 分頁 vs Code 分頁

	Chat 分頁	Code 分頁
成品在哪	對話裡預覽，按下載才有檔案	直接存在你電腦的資料夾
能做幾個檔	通常一個 HTML	整個專案，幾十個檔都行
能執行什麼	只能跑前端	能開伺服器、連資料庫、跑測試
關掉之後	對話還在紀錄裡，要用得自己下載	檔案還在，下次接著做
今天用在	第 1 章	第 2–5 章

提示｜Chat 分頁還有「Cowork」模式，適合整理文件、做報告；今天統一用 Code。

CONCEPT

網頁版、Desktop、終端機，差在哪

	網頁版	Desktop App	終端機（CLI）
打開	瀏覽器	安裝 App	安裝後打指令
操作	對話	點按鈕，有圖形介面	全程打指令
功能	只有對話（＝Chat 分頁）	Chat ＋ Code 分頁	Code（同一個引擎）
適合	沒裝 App 的電腦	**今天全程用這個**	習慣打指令的人

提示｜網頁版：https://claude.ai。三種用同一個帳號、同一份額度；做網站這些 Code 功能，只有 Desktop 和終端機有。

PROMPT

PTCF：把提示詞寫清楚的四字口訣

元素	說明	範例
Persona 角色	要 Claude 扮演誰	你是旅遊規劃師
Task 任務	具體要做什麼	規劃一趟三天兩夜的台南行程
Context 脈絡	背景、限制、給誰用	兩位長輩同行，不要走太多路
Format 格式	產出形式	每天一張表格，繁體中文

提示｜四個不一定都要寫。缺哪一個，AI 就自己幫你決定哪一個，你不滿意的地方，通常就是你沒寫到的那一格。

PROMPT

同一件事，差在哪？

模糊版本

幫我做一個購物網站

賣什麼？要不要付款、會員、後台？
Claude 只能猜，猜錯就要重來，
而且很可能做得太大。

PTCF 版本

- **P** 你是網頁設計師
- **T** 做一個賣文具的購物網頁
- **C** 6 件商品、可加入購物車、不串金流、不需要登入
- **F** 單一 HTML 檔、手機也好用、繁體中文

PROMPT

完整範例：文具小店購物網頁

P T C F 都有（示範用，不串金流）

你是網頁設計師。請做一個「文具小店」購物網頁，
單一 HTML 檔，不需要後端。
功能：
・首頁列出 6 件商品（圖片用色塊代替），
　每件有名稱、價格、「加入購物車」
・購物車可以改數量、刪除，即時顯示總金額
・結帳頁填收件資料，送出後顯示「訂單完成」
・不串金流、不需要登入；購物車資料暫存在瀏覽器
風格：手機也好用、配色清爽、繁體中文。
先給我計畫，我確認再動工。

提示｜試著找找看：哪一句是 P、T、C、F？

PREPARE

事先準備清單

項目	用在哪	狀態
Claude 帳號（學校 Team）	整場都會用到	現場加入
Vercel 帳號	第 2 章部署上線	請先註冊
OpenAI API key	第 3 章聊天機器人	現場提供
Chrome 瀏覽器	第 2、3、5 章	語音與 Playwright 會用到

提示｜Node.js、Git 這些工具，現場會一步一步帶你裝，不需要事先準備。

CHAPTER 01

Desktop 的 Chat 分頁

不用寫程式，先用對話做文字分析與數據分析

1-1 文字分析　1-2 數據分析

CHAT

這一章用 Chat 分頁

分頁	用來做什麼	今天用在
Chat	對話、貼文件問問題、整理內容	第 1 章
Cowork	Chat 分頁裡切換；整理文件、做報告	今天不用
Code	讓 Claude 動手改你電腦裡的檔案	第 2 章起

提示｜Chat 和 Code 是同一個帳號、同一份額度，不用另外開網頁。

1-1 TEXT

1-1　文字分析：讓 Claude 讀文件

摘要

把 20 頁壓成 1 頁重點

問答

針對文件內容問問題，並指出出處

比對

兩份文件哪裡不同

擷取

把資訊整理成表格

提示｜先從「摘要」和「問問題」開始，今天先看看 Claude 讀得懂到什麼程度。

1-1 TEXT

練習 1：讀一份 PDF

1　點「+」上傳 PDF

2　請 Claude 先摘要

3　接著針對內容問問題

4　每個答案都要求附上頁碼

第一個 prompt

請讀這份 PDF，用繁體中文給我：
1. 三句話的整體摘要
2. 五個重點（每點一行）
3. 這份文件沒有提到、但讀者可能會想問的問題

提示｜範例資料為青少年憂鬱症衛教資料，也可以用自己想用的資料嘗試。

1-1 TEXT

PDF 練習：可以問的問題

- 這份資料如何描述青少年憂鬱症常見的表現？
- 資料中提到哪些可能的原因或風險因素？
- 家人或老師可以怎麼支持？請只根據文件內容回答
- 有哪些情況文件建議要尋求專業協助？
- 把全文整理成一張「症狀／原因／支持方式」對照表

注意｜最後一題是重點：要求 Claude **只根據文件**回答，並附頁碼。

Claude 怎麼「讀」檔案：三種方式

1　直接讀

模型直接讀內容；文件很長時，可能只挑相關段落。適合查條款、摘要、比文字。小心：不一定逐字讀完

2　寫程式來算

打開「程式執行」後，Claude 自己寫 Python 讀檔、計算、產出 Excel。適合加總、比對、合併。好處：能重跑

3　用眼睛看

把圖片或 PDF 頁面交給模型「看」，像 OCR 加上看版面。適合掃描檔、發票、照片。小心：數字可能看錯

提示｜用哪一種是模型自己決定。所以 prompt 要寫明「用什麼方法」，並要它「回報」讀了什麼。

1-1 TEXT

練習 2：讀懂表單

- 文件是**虛構的假案例**，不涉及任何真實個人
- 表單有勾選、長段文字、小表格；要驗證的是**它讀對了沒有**

直接問問題

這是一份個案資料表單，請根據文件回答：

1. 個案的經濟來源為何？
2. 個案的家庭成員有誰？
3. 個案是否需要使用輔具？若需要的話，請告訴我是使用哪些輔具？
4. 個案的身心障礙原因為何？
5. 個案期望可以改變的目標是什麼？

提示｜每題都回原文件核對一次，看它答得對不對。

1-1 TEXT

表單常見的難讀之處

勾選符號

■ □ 之外，有些選項沒有符號，看不出有沒有勾

一格多筆

輔具、服務項目、協助內容擠在同一格

格子裡的小表格

輔具那一塊其實是一張勾選矩陣

長段敘述

整格都是文字，重點藏在句子裡

空白欄位

姓名、日期沒填，不代表資料遺失

勾選與敘述不一致

勾「無」，下面卻寫了內容

提示｜這些都是真實表單很常見的樣子。剛才答錯的地方，多半就出在這裡。

1-1 TEXT

怎麼確認 Claude 讀對了

- **逐題核對**：每個答案都回原文件看一次
- **看有沒有漏項**：例如輔具有幾種、家庭成員有幾位
- **要它標出不確定**：叫它列出讀不準的地方
- **反過來問**：「輔具欄你讀到哪些？」再回原文核對

> **注意**｜表單最容易出錯的是勾選被漏掉、一格多筆被當成一筆、空白被腦補成內容。

1-1 TEXT

表格讀不好時，可以怎麼處理

1. **先複述結構**
   先確認它看到了什麼
2. **再整理成乾淨清單**
   一個欄位一行，勾選項標明「已勾／未勾／無法判斷」
3. **最後再問問題**
   拿整理好的清單去問

步驟 1 prompt

```
請先不要分析，只告訴我：這份文件分成哪幾個區塊、
每個區塊有哪些欄位、哪些勾選項你無法確定有沒有勾？
請列出來，不要猜。
```

1-1 TEXT

整理成乾淨清單

步驟 2 prompt

請把這份表單整理成「欄位：內容」的清單，依區塊分組。
勾選項請標註「已勾／未勾／無法判斷」，空白欄位請寫「（空白）」。
如果有任何一格你不確定怎麼讀，請標註「不確定」，不要猜。

提示｜先整理、再提問，通常比直接問原始表格更準。

1-1 TEXT

PDF：先分清楚是文字型還是掃描型

文字型 PDF

從 Word、ERP、報表系統直接匯出。用程式直接讀出文字和表格。小心跨頁表格、雙欄的閱讀順序

掃描型 PDF

紙本掃描或手機拍照，其實是一張圖。讓模型看圖認字（OCR）。小心數字認錯、印章蓋到、框線斷掉

提示｜怎麼分：請它抓第一頁的文字，抓不到或亂碼，就是掃描型。

1-1 TEXT

練習 3：掃描型 PDF 發票

- 資料：掃描型 PDF，3 頁各一張發票（上傳到 Chat）
- 要抓：號碼、日期、統編、銷售額、稅額、總計
- 檢查：總計請它用程式重算，對不上的列出來

Prompt（可直接複製）

附件是掃描的統一發票，一頁一張。請逐頁看圖，整理成表：
頁碼｜發票號碼｜日期｜賣方統編｜銷售額｜營業稅｜總計
1. 看不清楚的字寫「?」，不要猜。
2. 照發票原樣抄，不要修正任何數字。
3. 用 Python 重算 銷售額＋營業稅＝總計，列出不符的頁。

小結｜紙本發票人工打字慢又容易錯；認出來之後，還要能抓出發票本身的錯。

1-1 TEXT

處理真實文件：Chat 與 Code 差在哪

	Chat 分頁	Code 分頁（第 2 章）
檔案怎麼給 Claude	你一份份上傳	選一個電腦資料夾，它在資料夾裡讀
內容會送給 AI 嗎	會	也會（讀到的內容一樣要送給模型處理）
差別在哪	操作簡單	不用一份份上傳，且你決定它能讀哪個資料夾

注意｜改用 Code 分頁並不會讓資料「留在自己電腦」，內容仍會被送去處理。真實文件一律先**去識別化**。

1-1 TEXT

文件分析，還可以做什麼

出小測驗

十題選擇題，附解答

換個說法

改寫成給國中生、長輩、主管看的版本

找矛盾

文件前後有沒有互相衝突的地方

兩份比對

新舊版本哪裡不一樣

擬大綱

整理成一頁簡報大綱

翻譯與校對

轉換語言、檢查錯字與語病

1-2　數據分析

- 今天有兩個檔案：`sales_2026H1.csv`（銷售明細）、`stores.csv`（門市資料）
- 資料**不乾淨**：這正是真實世界的樣子
- 目標：診斷 → 清理 → 分析 → 交叉核算 → 儀表板

1. **先診斷**
   叫它挑毛病，不要叫它分析
2. **再分析**
   照它自己開的 SOP 清理後分析
3. **最後驗證與視覺化**
   換對話交叉核算，再做儀表板

1-2 DATA

先診斷：叫它挑毛病，不要叫它分析

新對話，上傳兩個 CSV 後貼上

你是零售業資深資料分析師。這是山嵐咖啡 2026 上半年的
銷售明細與門市資料。
請先做資料品質檢查：缺值、重複列、異常值、欄位值不一致
等問題，一項一項列出來，並說明每個問題你建議怎麼處理。
先不要做任何分析。

注意｜最後那句是關鍵：不擋住它，它會直接用沒清乾淨的資料算出漂亮數字，這比沒有結論更危險。

髒資料常見的樣子

類型	這份資料可能有
拼寫不一致	台北／臺北、簡繁混雜
退貨	數量是負數的交易
極端值	單價多打了幾個零
缺值與重複	空白欄位、同一筆出現兩次

提示｜不管有沒有程式背景，每個人都會遇到髒資料。

1-2 DATA

它列完之後，要它拿出證據

追問：證據、方法、不確定

針對你列出的問題，挑出影響營收最大的三項：

1. 每一項給我 3 筆原始資料列當證據
2. 附上你用來找出它們的 Python 程式碼
3. 哪一項你最不確定？為什麼？

提示｜它列得很完整，不代表都對。要它拿出原始資料和程式碼，你才有辦法自己對一次；問它最不確定什麼，往往會問出它自己沒說的假設。

1-2 DATA

照它開的 SOP 清理，然後分析

留在同一個對話

請依你剛才建議的 SOP 清理資料，然後進行以下商務分析：
1. 每個門市的各月營收趨勢，哪間門市成長最快？
2. 產品組合是否有季節性（月份）變化？
3. 平日與週末的消費行為與客單價有何差異？
4. 結合 stores.csv 的坪數，計算各門市每坪月營收（坪效），
   哪間店最好？

請務必列出分析結果的具體依據數字。

提示｜最後一句讓它把數字攤開：沒有數字的結論無從驗證。

交叉核算：換一個新對話

1. **先在原對話問**
   「只計算『高雄夢時代店』2026 年 6 月總營收，並列出算式。」

2. **開全新對話，只上傳 sales_2026H1.csv**
   貼上同一句話

3. **比對兩邊數字**
   不一樣就把兩邊的算式並排，找出是哪幾筆處理方式不同

小結｜數字不一樣不代表誰錯了，通常是定義差異：例如單價 9999 的那筆，一邊改回牌價算進去、一邊直接排除。把差異問出來，你才知道那個數字是什麼意思。

1-2 DATA

做成一頁主管儀表板

貼進對話

請將上述分析結果做成一頁互動式 HTML 儀表板 Artifact：

- 最上方放 4 張重點指標卡（總營收、訂單數、客單價、坪效最佳店）
- 中間放月營收趨勢圖與產品類別銷售佔比圓餅圖
- 下方放你認為最重要的一個經營洞察，用一句話寫給主管看
- 繁體中文，風格參考《經濟學人》圖表的簡潔現代感

提示｜「一句話寫給主管看」在逼它取捨，挑出最重要的一個，才是分析師的價值。

1-2 DATA

驗收，然後繼續追問

- 指標卡上的數字，和你核算過的結果**一致**嗎？
- 圖表看得懂嗎？有沒有哪張你不知道在說什麼？（那就問 Claude）
- 追問：「加一個門市下拉選單，切換時趨勢圖跟著更新」
- 追問：「加月份篩選按鈕，指標卡與圓餅圖一起連動」

> **注意**｜AI 給的「策略建議」是它猜的。它不知道你的成本結構，也不知道那間店為什麼坪效差。拿去參考可以，拿去做決策不行。

1-2 DATA

數據分析，還可以做什麼

預測趨勢

下個月大概會賣多少

找異常

哪一天、哪一家的數字不尋常

分群

把顧客或門市分成幾種類型

寫成報告

把發現整理成給主管的一頁摘要

做成簡報

挑三張圖加結論，轉成簡報大綱

換資料再試

拿你自己領域的表格也試試看

TRANSITION

Chat 的天花板

- 想拆成 `index.html`、`style.css`、`app.js`：通常只給你一個 HTML
- 想加後端存資料：瀏覽器裡跑不了伺服器
- 想讓它直接改你電腦裡的檔案：Chat 做不到
- 開新對話就得整份貼回去

> **接下來**｜所以下一章，我們換到 Code 分頁，讓 AI 在你的電腦裡動手。

CHAPTER 02

貪食蛇與語音轉文字網站

裝好環境，用 Plan Mode 做貪食蛇練習改需求；再自己走一輪，做語音轉文字網頁，都部署上線

環境安裝　Plan Mode　貪食蛇　語音網頁　部署

SETUP · NODE.JS

步驟 1：安裝 Node.js

1. **到 https://nodejs.org**
   下載標示 **LTS** 的安裝檔（Windows 選 .msi，Mac 選 .pkg）。LTS 是長期支援版，不要選 Current

2. **雙擊執行，一路 Next**
   保持預設。若問要不要裝額外編譯工具，不用勾

3. **關掉所有終端機，重開一個新的**
   PATH 只有新視窗才讀得到

4. **驗證**
   終端機輸入 `node -v` 和 `npm -v`，有版本號就對了

SETUP • GIT

步驟 2：安裝 Git

步驟	Windows	Mac
安裝	到 https://git-scm.com/downloads/win，下載 64-bit Setup，一路 Next	終端機輸入 `git --version`，跳出視窗就按「安裝」
驗證	新開 PowerShell，輸入 `git --version`	裝完再輸入一次 `git --version`

順手設定一次（兩個系統都一樣）

```
git config --global user.name "你的名字"
git config --global user.email "你的信箱"
```

SETUP

自檢：全部過才往下走

檢查	指令或動作	正確的樣子
Node.js	`node -v`	有版本號
Git	`git --version`	有版本號
Desktop	開 App → Code 分頁	能選資料夾、能對話
CLI	`claude --version`	有版本號

提示｜卡住時把整段錯誤訊息複製，貼到 https://claude.ai 問它，這就是 vibe coding 的第一課。

DESKTOP

第一次對話：確認環境沒問題

貼進對話區

你好，請用一句話告訴我你現在的工作資料夾是哪裡，
並在裡面建立一個 `hello.txt`，內容寫「工作坊開始」。

- Claude 會要求建立檔案，這時按「同意」
- 資料夾裡出現 hello.txt，就代表一切正常
- 資料夾路徑盡量用純英文

PROJECT

今天要做的：貪食蛇

- 一個**貪食蛇遊戲網站**，做完部署上線，用網址就能玩
- 重點不是做出貪食蛇，是練習**改需求**：先做基本版，再一步步加變形
- 拆成 `index.html`、`style.css`、`game.js` 三個檔：這是 Chat 做不到的

> **提示**｜貪食蛇的規則大家都懂，所以你可以專心看：要怎麼講，Claude 才會做出你要的東西。

CONCEPT

什麼是「靜態網站」

- 只有 HTML、CSS、JavaScript，**不需要後端伺服器**
- 打開檔案就能玩；很容易放到網路上
- 適合：小遊戲、測驗、簡介網頁、作品集
- 不適合：需要登入、儲存使用者資料、串接 API key

> **提示**｜今天這一章做靜態網站，第 3 章才會加上後端。

PLAN MODE

Plan Mode 到底是什麼

1. **Claude 只規劃、不動手**
   你看到的是計畫，不會有任何檔案被改

2. **你先檢查、先修正**
   方向不對，在計畫階段改最便宜

3. **確認之後才動工**
   不用重來，只要在對話裡修正計畫

> **提示**｜Plan Mode 的價值：把「做錯方向」的成本壓到最低。

PROMPT

基本版 prompt

Plan Mode

在這個資料夾做一個貪食蛇遊戲，拆成 index.html / style.css / game.js 三個檔案。
規則：
・20 × 20 的格子，用 canvas 畫
・方向鍵控制，蛇不能瞬間回頭（往右時按左要無效）
・吃到食物長一節、分數 +1，食物換位置
・撞牆或撞到自己 → 遊戲結束，畫面顯示分數和「按空白鍵重來」
・食物不可以出現在蛇身上
畫面上方顯示目前分數和最高分。
先不要寫程式碼，給我計畫；做完再告訴我怎麼打開來玩。

提示｜每一條規則都是在替 Claude 做決定。沒寫的部分，它就自己猜。

PLAN MODE

開 Plan Mode，貼上 prompt

1. **開一個新資料夾**
   例如 snake，路徑用英文
2. **權限選單切到「計畫（Plan）」**
   在輸入框下方，選資料夾旁邊
3. **貼上基本版 prompt 並送出**
4. **等 Claude 給出計畫，先不要同意動工**

PLAN MODE

讀計畫時看這四件事

檔案

三個檔各自負責什麼？

規則

格子大小、速度、怎麼算撞到？

結束與重來

分數、最高分怎麼處理？

待決定

它有沒有問你問題？

提示｜不滿意就直接說：「速度慢一點」「最高分要記在瀏覽器裡」，它會更新計畫。

BUILD

確認計畫，開始建置

- 回答 Claude 提出的待決定事項
- 覺得 OK 就同意計畫，讓它開始動工
- 做完請它告訴你怎麼打開來玩，或直接在 App 的預覽裡玩

> **提示**｜動工要一點時間，趁空檔想想：等一下要怎麼找碴。

REVIEW

驗收：玩五分鐘，專門找碴

- 快速連按兩個方向鍵，蛇會不會自己撞死
- 蛇很長的時候，食物有沒有生在身體上
- 遊戲結束後按空白鍵，分數有沒有歸零
- 連續重來三次，速度會不會越來越快

找到問題怎麼講：症狀＋期待

遊戲結束後按空白鍵重來，
蛇的移動速度變成兩倍。
再重來一次又更快。
每次重新開始速度應該
都跟第一局一樣。

提示｜改到一半發現方向錯了，按 `Esc` 停下；想退回上一步，輸入 `/rewind` 。

CHANGE

變形一：穿牆模式

Prompt

加一個「穿牆模式」的開關（畫面上的核取方塊）。
打開時：蛇撞到牆不會死，會從對面那一邊冒出來。
關閉時：維持現在撞牆就結束的規則。
注意：
・穿牆的時候蛇身要連續，不可以斷成兩截
・撞到自己還是會死，這個不受開關影響
・遊戲進行中切換開關要立刻生效，不用重開一局

小結｜「加一個穿牆模式」底下藏了三個 AI 猜不到的決定：撞到自己算不算死？切換要不要重開？穿過去怎麼畫？把模糊地帶先講掉，是提示詞最值錢的部分。

CHANGE

變形二、三：雙食物＋暫停

變形二：兩顆食物

場上同時要有兩顆食物：

- 紅色 = 一般食物，+1 分、長一節
- 金色 = 特殊食物，+5 分、長三節，出現 8 秒後消失，過 5 到 10 秒隨機再出現一顆

兩顆食物不能重疊，也不能生在蛇身上。

變形三：暫停

按 P 或空白鍵可以暫停／繼續。

暫停時：

- 蛇完全靜止，方向鍵無效
- 畫面正中央顯示「已暫停」
- 金色食物的倒數也要跟著停

遊戲結束後按暫停鍵應該沒反應，不要跟重新開始的功能打架。

提示｜變形三的最後一句是重點：空白鍵原本是「重新開始」，不先講清楚，你會得到一個按了就重開的暫停鍵。

PRACTICE

自由練習：挑兩個做做看

難度	可以加的功能
★	難度選單（慢／中／快）、吃到食物有音效、深色／淺色主題
★★	障礙物（撞到就死）、60 秒計時挑戰、手機觸控滑動操作
★★★	雙人對戰（WASD vs 方向鍵）、最高分排行榜（重新整理還在）、會自己走的 AI 蛇

提示｜每次只加一個功能，玩過、確認沒壞，再加下一個。

DEPLOY

什麼是部署

- 本機的檔案只有你自己看得到
- **部署**＝把網站放到網路上，任何人用網址就能打開
- 靜態網站部署最容易，也不用付費

DEPLOY

用 Vercel 部署：流程

1. **確認已有 Vercel 帳號**
2. **對 Claude 說**
   「請幫我把這個遊戲部署到 Vercel，部署完成後告訴我網址。」
3. **登入授權**
   瀏覽器會跳出視窗，登入並同意
4. **取得網址**
   Claude 會回報，打開網址玩一次

DEPLOY

部署之後怎麼確認

- 打開網址，完整玩一次；想用手機玩，記得先做「觸控滑動操作」
- 把網址傳給旁邊的人，請對方玩
- 要更新？回到 Claude 修改，再說一次「重新部署」

注意｜只部署靜態網站。第 3 章的聊天機器人有後端和 API key，今天不上線。

PROJECT 2

第二個作品：語音轉文字網頁

- 概念像語音輸入工具：**講話，文字就出現在畫面上**
- 用瀏覽器內建的語音辨識（Web Speech API）：**免費、不用 API key**
- 一樣是靜態網站，拆成 `index.html`、`style.css`、`app.js`，做完可以部署
- 這次換你自己走完整流程：**計畫 → 建置 → 找碴 → 改需求 → 部署**

> **注意**｜今天請用**電腦**的 **Chrome 或 Edge**。手機（尤其 iPhone）和 Safari、Firefox 多半不支援瀏覽器內建的語音辨識。

PROMPT

語音版 prompt

Plan Mode

在這個資料夾做一個「語音轉文字」網頁，拆成 index.html / style.css / app.js。
・用瀏覽器內建的語音辨識（Web Speech API），語言設繁體中文，不用 API key
・畫面中間一個大麥克風按鈕：按一下開始聽，再按一下停止；聽的時候按鈕變紅
・說話時，辨識中的文字用灰色即時顯示；一句確定後，加進下方文字框
・文字框可以自己修改，旁邊有「複製」和「清除」兩個按鈕
・瀏覽器不支援、或使用者拒絕麥克風權限時，畫面上要顯示清楚的提示
先不要寫程式碼，給我計畫；做完告訴我怎麼在本機測試。

提示｜「按一下開始、再按一下停止」「灰色是還沒確定的字」都是在替 Claude 做決定。

PLAN MODE

開資料夾、讀計畫

計畫裡看	問自己
檔案	三個檔各負責什麼？
開始與停止	按鈕怎麼切換？停頓太久會不會自己停？
文字	辨識中（灰色）和確定的字怎麼分？
出錯	瀏覽器不支援、沒給麥克風權限時，畫面顯示什麼？

提示｜開新資料夾 **voice-note** → 權限切到「計畫」→ 貼上 prompt → 讀完計畫、回答待決定事項，再同意動工。

TEST

本機測試：要用 localhost 打開

- 直接雙擊 `index.html`，麥克風**可能不能用**
- 請 Claude：「幫我開一個本機預覽」，用它給的 `http://localhost:…` 網址打開
- 瀏覽器問能不能使用麥克風，按「允許」
- 按下麥克風講一句話，看文字有沒有出現

> **提示**｜按了沒反應？看網址列左邊的麥克風圖示，是不是被封鎖了。

REVIEW

驗收：講三分鐘，專門找碴

- 講一段長句，中間停頓三秒，會不會自己停掉
- 中英夾雜：「明天要交 report 給老師」
- 按停止再按開始，前面的文字還在不在
- 拒絕麥克風權限，有沒有出現提示

找到問題怎麼講：症狀＋期待

我講話停頓幾秒，
它就自動停止了，
按鈕卻還是紅色的。
希望我按停止之前
都一直聽，按鈕顏色
也要跟實際狀態一致。

提示｜跟貪食蛇一樣：一次講一個問題，修好再找下一個。

CHANGE

改需求：讓它更好用

變化一：用說的加標點

說「逗號」「句號」「問號」「換行」時，
不要打出這兩個字，改成對應的
標點符號或換行。
只處理確定的文字，
灰色的即時文字不用轉。

變化二：歷史紀錄

每次按停止，把這段文字存成一筆紀錄，
顯示在右邊的「歷史」列表：

- 顯示時間和前 20 個字
- 點一下就把整段放回文字框
- 重新整理頁面，紀錄還在
- 每筆旁邊有刪除按鈕

提示｜每個 prompt 都講清楚「什麼時候要、什麼時候不要」，這是改需求最重要的一招。

DEPLOY

部署，再請同學試

1. **對 Claude 說**
   「請幫我把這個網頁部署到 Vercel，完成後告訴我網址。」

2. **用電腦打開網址**
   允許麥克風，講一句話，看文字有沒有出現

3. **把網址傳給旁邊的同學**
   請他也用電腦的 Chrome 或 Edge 試試看

注意｜手機不一定能用，尤其 iPhone：手機瀏覽器對內建語音辨識的支援很差，今天不用手機測。想在手機上穩定使用，要改用 Whisper，但它是錄完才轉，不是即時。

CHALLENGE

本章小挑戰

基本版

貪食蛇和語音網頁都能用、找碴都通過，且都部署成功

進階

貪食蛇做完兩個變形；語音網頁做完兩個變化

之後｜網頁只能把字打在自己的頁面上。有時間的話，課程最後（第 122 頁）會把語音工具包成**桌面 App**，按快捷鍵就能把字貼進 Word、LINE。

RECAP

第 2 章回顧

- **環境**：Node.js、Git、Claude Desktop 全部就位
- **Plan Mode**：先看計畫，再動工
- **改需求**：把模糊地帶先講清楚，一次加一個功能
- **完整流程**：語音網頁自己走完一輪
- **部署**：靜態網站很快就能上線

> **接下來**｜下一章要做 RAG 聊天機器人，需要 API key。

CHAPTER 03

動態網站：RAG 聊天機器人

加上後端、API key、資料庫，讓機器人根據你的資料回答

成品：會查資料、附出處、會記憶的機器人（本機）

CONCEPT

靜態網站 vs 動態網站

	靜態（第 2 章）	動態（第 3 章）
有沒有後端	沒有	有
能不能存資料	只能存在自己的瀏覽器	能（存在資料庫）
能不能用 API key	不能（會外洩）	能（key 放後端）
部署	很容易	較複雜，今天只在本機

RAG

RAG 是什麼：開書考試

- 一般聊天機器人：憑記憶回答，不知道的也可能自己編
- **RAG**：先翻你給的資料，找到相關段落，再根據它回答

1. **把資料切成小段**
   例如衛教 PDF 每幾段一組
2. **找出和問題最相關的段落**
   像在書裡翻到對的那幾頁
3. **把段落和問題一起交給模型**
   只根據這些段落回答，並附上出處

ARCHITECTURE

RAG 聊天機器人的架構

1　瀏覽器

使用者打字、看回覆

2　後端

保管 API key、找出相關段落

3　OpenAI API

根據段落產生回覆

4　SQLite

存資料段落和對話

提示｜瀏覽器**不直接**呼叫 OpenAI，一律經過後端，key 才不會外洩。

API KEY

API key 是什麼，為什麼要保護

- API key ＝ 你的帳號通行證，**誰拿到就能花你的錢**
- 放在 `.env` 檔案裡，**不要**寫進網頁程式碼
- 不要貼到群組、截圖、上傳到 GitHub
- 今天的 key 由老師提供，已設好**用量上限**

> **注意**｜如果懷疑 key 外洩，立刻到後台刪除並重新產生。

API KEY

領取今天的 API key

1. **複製 key**
   貼到一個只有你看得到的地方，等一下建置時會用到

2. **不要外傳**
   不要傳給別人、貼到網路上或截圖分享

注意｜這把 key 只供今天課堂使用，課後會關閉。之後想繼續用，請自行到 OpenAI 申請。

DATABASE

SQLite：一個檔案就是資料庫

- 不用另外安裝資料庫伺服器，一個 `.db` 檔案就是整個資料庫
- Claude 會用 npm 套件 `better-sqlite3` 幫你處理
- 我們用它來存**資料段落**和**每一則對話**，重新整理之後歷史還在

> **提示**｜SQLite 適合本機練習；要放上網路的話，通常會換成雲端資料庫（例如 Supabase）。

DATA

選一份資料當知識庫

預設

第 1 章練習 1 的衛教 PDF，放進這次的專案資料夾

換成自己的

課程講義、規章、產品說明……選**文字型** PDF

- 掃描型 PDF 要先轉成文字，不然機器人讀不到
- 記得先想：它**幫誰**回答、**不能做什麼**

注意｜換自己的資料時，一樣不要用真實個資；去識別化之後再放進來。

PROMPT

基本版 prompt（Plan Mode）

Plan Mode

```
我想做一個 RAG 聊天機器人網頁，只在自己的電腦上跑，不需要部署。
・知識庫：資料夾裡的衛教 PDF（之後可能換成別的 PDF）
・用 OpenAI 的 gpt-5-mini 回答，API key 放在 .env
  （你幫我建好 .env，我只負責把 key 貼進去）
・只根據 PDF 內容回答並附頁碼；資料裡沒有就說「資料中沒有提到」
・不做診斷；使用者提到想傷害自己時，提供 1925 安心專線
・對話存進 SQLite，重新整理後還在
・盡量不要另外安裝資料庫或大型套件；介面簡潔、繁體中文
先不要寫程式碼，給我計畫：檔案怎麼分工、
要我先準備什麼、有什麼要我先決定的。
```

提示｜換自己的資料時，把第二行和第五行換成你的資料與注意事項。

PLAN MODE

讀計畫時看這四件事

- 有沒有把 API key 放在後端（`.env`）？
- PDF 怎麼切段、存在哪裡？
- 回答有沒有附出處？資料裡沒有時怎麼回？
- 它要我先準備什麼？有沒有要另外安裝的東西？

> **注意**｜如果計畫把 API key 寫進前端，要請它修正，這是不能妥協的。

BUILD

建置與放入 API key

1. **同意計畫，讓 Claude 開始建置**
2. **Claude 會幫你建好 `.env`**
   打開它，把 API key 貼在 `OPENAI_API_KEY=` 後面，存檔
3. **啟動後端**
   Claude 會幫你執行，並告訴你網址（通常是 localhost）
4. **用瀏覽器打開，開始聊天**

> **注意**｜.env 這個檔案只有你自己看得到，不要截圖分享。

REVIEW

驗收：基本版完成了嗎

- 問**資料裡有的**：答對，並附上頁碼（回 PDF 對一下）
- 問**資料裡沒有的**：會說「資料中沒有提到」，不會自己編
- 重新整理後，歷史對話還在
- API key 沒有出現在網頁原始碼裡

ADVANCED

進階 1：設定機器人的個性

貼進對話區

```
請幫機器人加上角色設定（system prompt）：
它是_____，說話風格_____。
遇到不確定的事，要誠實說不知道。
不要提供醫療／法律的確定診斷或結論。
```

提示｜角色設定決定機器人的「靈魂」，寫得越具體越好。

ADVANCED

進階 2：看過去的對話

- 左邊加一個「歷史對話」列表
- 點一下就能回到那一次對話
- 可以刪除單次對話

> **提示**｜這一步會用到資料表設計：一次對話包含很多則訊息。

VOICE

進階 3：語音輸入，兩條路

	Web Speech API	Whisper（OpenAI）
費用	免費	按用量計費（很便宜）
速度	即時	錄完再送出，稍有延遲
瀏覽器	以 Chrome / Edge 為主	各瀏覽器都能用
金鑰	不需要	與聊天共用同一把 key

提示｜第 2 章用的是瀏覽器內建（Web Speech）。這裡選 **Whisper**：和聊天共用 API key，不挑瀏覽器，比較穩。

VOICE

語音輸入 prompt

Plan Mode

請在聊天機器人加上語音輸入：
- 輸入框旁加一個麥克風按鈕，按一下開始錄音、再按停止
- 錄好的聲音送到後端，由後端呼叫 OpenAI 的 Whisper 轉成文字
- 轉出來的文字先放進輸入框，讓使用者確認後再送出
- 支援繁體中文

先給計畫，我確認再動工。

VOICE

語音常見問題

症狀	可能原因
按了沒反應	瀏覽器沒給麥克風權限，網址列左邊檢查
一直失敗	必須在 localhost 或 https 才能用麥克風
轉出來是亂碼	確認有指定語言為中文
很慢	錄音太長，先錄短一點

提示｜遇到錯誤時，把畫面上的錯誤訊息整段貼給 Claude，它通常能自己修。

NEXT STEP

為什麼今天不部署這個作品

- SQLite 的檔案在 Vercel 這類平台上**不會保留**，資料會消失
- 要上線，需要換成**雲端資料庫**（例如 Supabase），並妥善保管 API key
- 所以今天只在自己的電腦上跑，把重點放在「做出會運作的東西」

> **提示**｜日後想試：Supabase 是 Claude 的官方連接器。在連接器裡搜尋 Supabase 連上後，就能請 Claude 幫你建資料表、查資料。

CHALLENGE

本章小挑戰

基本版

根據 PDF 回答並附出處、有歷史紀錄、key 在 .env

進階

角色設定、語音輸入、歷史對話列表

提示｜先做基本版。語音輸入是進階題，時間不夠可以不做。

RECAP

第 3 章回顧

- **RAG**：先查資料再回答，並附出處
- **後端**的作用：保管 key、找段落、存資料
- **SQLite**：一個檔案就是資料庫
- **Whisper**：把聲音變文字；動態網站今天只在本機

CHAPTER 04

Skill：把好用的方法存起來

做一次、喜歡它、請 Claude 存成 Skill，再開新對話測試

成品：一個你自己的 Skill

資料夾整理

實作：資料夾整理（動機與資料）

- 情境：專案資料夾裡的 `folder` 有 9 個檔案，檔名看不出內容，還有重複的
- 例：`合約_達豐企業(1).docx`、`合約_達豐企業_final.docx`、`掃描0001.pdf`、`IMG_2031.jpg`

問題

檔名看不出內容、有重複檔

規則

只能複製，不能刪或改原始檔

提示｜開 **Code 分頁**新對話，選**專案資料夾**（裡面有 `folder`）。權限選「手動（Manual）」，Claude 動檔案前會先問你。

資料夾整理

實作：資料夾整理（Prompt）

Prompt

請整理資料夾「folder」，不要刪除或修改任何原始檔：
1. 用 Python 列出每個檔案的檔名、大小、日期、SHA-256，相同的標為重複。
2. 讀內容判斷類型、對象、日期（民國年換西元），依「日期_類型_對象」提新檔名。
3. 先列表給我確認，確認前不要動任何檔案。
4. 確認後建「整理後/」並複製，存檔案索引 Excel，回報控制總數。

提示｜驗算：原始檔案數 = 整理後檔案數 + 重複檔數。

資料夾整理：參考答案

原檔名	新檔名／結果
合約_達豐企業_final.docx	20250305_合約_達豐企業.docx
合約_達豐企業(1).docx	重複（SHA-256 相同）
新增 Microsoft Word 文件.docx	20250410_合約_宏祺商行.docx
掃描0001.pdf	20250512_發票_永捷資訊.pdf
IMG_2031.jpg	20250515_現場照片_人孔蓋.jpg

提示｜節錄 5 個。結果略有不同沒關係，重點是：沒有檔案被刪、數量對得起來。

SKILL

Skill 是什麼，什麼時候才需要

- Skill ＝ 寫給 Claude 的**做事手冊**。做一次，之後每個新對話都能用
- 平常 Claude 只看得到它的名稱和簡介，需要時才讀全文
- 判斷標準：我用一句話講，Claude 做出來會一樣嗎？

不需要寫成 Skill	值得寫成 Skill
部署到 Vercel、改個顏色：一句話就講得清楚	你的固定格式、做事順序、判斷標準：每次都要重講

SKILL

先做，再存起來

1. **先挑一個任務，跟 Claude 做一輪**
2. **看到喜歡的做法，先別換話題**
3. **請 Claude 把它存成 Skill**
   存的是**做法**（格式、順序、檢查標準），規則 Claude 會替你歸納
4. **開新對話，丟同類的題目，看它有沒有照做**

存成 Skill 的說法

我很喜歡剛剛____的方式。
請把它整理成一個 Skill：以後我遇到____這類問題，都照這樣做。

Skill 長什麼樣子

一個 Skill = 一個資料夾

```
folder-organizer/
├── SKILL.md    ← 步驟和規則
├── scripts/    ← 用過的程式
└── tests/      ← 測試用檔案
```

SKILL.md（示意）

```
---
name: folder-organizer
description: 使用者要整理資料夾、
  重新命名檔案時使用。
---
1. 列出檔案與 SHA-256，標出重複
2. 讀內容，提新檔名
3. 先列表，等使用者確認
4. 只複製，不刪除原始檔
```

提示｜Skill 其實就是文字檔。`description` 決定 Claude 什麼時候用它；看得懂，就能自己改。

SKILL

實作：把整理流程存成 Skill

Prompt

請把剛剛做對的資料夾整理流程做成一個 Skill：
1. 名稱 folder-organizer，說明寫清楚什麼時候使用。
2. SKILL.md 寫步驟和規則：不得刪除、先列計畫等我確認。
3. 用過的程式存到 scripts/，資料夾路徑不要寫死。
4. 存成我的個人 Skill，讓所有資料夾都能用。

提示｜存好後輸入 `/skills` ，清單裡有 folder-organizer 就成功了。

怎麼建立與使用 Skill

步驟	做什麼
1 先做對	把任務做到結果正確
2 存成 Skill	請 Claude 整理，不用自己寫
3 換資料測	開新對話，不重貼 prompt
4 打開看過	SKILL.md 有沒有寫到重點、有沒有多寫
5 直接叫用	說明寫得好，Claude 會自己用

提示｜第 3 步最重要：不重貼 prompt 也能做對，才算存成功。

SKILL

實作：換新資料測試 Skill

- `folder_test` 資料夾有 5 個檔案（`文件1.docx`、`scan_20250618.pdf`、`發票-華揚.pdf` …）
- 陷阱：兩個 PDF 內容一模一樣
- 目的：換新資料，看 Skill 能不能照流程做

Code 分頁開新對話，選 folder_test 資料夾，只說這一句

整理這個資料夾

Skill 測試：參考答案

原檔名	新檔名／結果
文件1.docx	20250602_合約_華揚工程.docx
scan_20250618.pdf	20250618_發票_華揚工程.pdf
發票-華揚.pdf	重複
Book1.xlsx	202506_費用明細.xlsx
新文件 (3).docx	20250616_驗收紀錄_華揚工程.docx

提示｜沒照流程做（例如沒先列表確認），就回去修 SKILL.md 的步驟。

有沒有 Skill，差在哪

對照	沒有 Skill	有 Skill
開頭	直接開始改名	先列計畫，等你確認
重複檔	不一定發現	用 SHA-256 標出來
原始檔	可能被改到	只複製，不刪不改
收尾	不一定回報	回報控制總數

提示｜差別不是 Claude 變聰明，是它每次都照同一份手冊做。

SKILL

第二個例子：山嵐咖啡

- 第 1 章做過：先診斷、再清理、再分析、最後交叉核算
- 該存的不是這份資料，而是「怎麼確保分析可信」的**做法**

存成 Skill

我很喜歡剛剛分析資料的做法：
先做資料品質檢查、不擋住就不分析、
結論一定附具體數字、最後用新對話交叉核算。
請整理成一個 Skill，名稱「資料分析 SOP」。

提示｜同一招可以存任何「每次都要重講」的做法：格式、順序、檢查標準。

SKILL

換你練習

- 自由發揮，或跟著下表做；不另外提供資料，用你自己現有的資料就可以

領域	可以做的事	存下來的是什麼
醫療	把衛教資料整理成病人看的單張	輸出格式
法律	檢查一份契約或條文	檢核流程
商管	寫一封客戶信，或分析一份銷售資料	寫作風格／分析 SOP
人文／論文	讀一段文獻	提問引導（先講重點，再問你問題）

注意｜不要用真實個資、病歷、合約；用假資料，或已去識別化的資料。

SKILL

存完要檢查

- 打開 Claude 寫的 Skill 看一遍：有沒有寫到你要的重點？有沒有多寫？
- 看「什麼時候使用」那句簡介：太籠統會亂觸發，太窄會叫不出來
- 不滿意，直接用對話修改：「再加一條，不要用專業術語」

> **注意**｜Skill 是 Claude 歸納的，可能歸納錯或過度。驗收的原則一樣適用。

別人寫好的 Skill：去哪找

- 除了自己寫，網路上有很多寫好的 Skill，可以直接拿來用
- Anthropic 官方的 Skill 都放在 GitHub：`https://github.com/anthropics/skills`

Skill	做什麼
frontend-design（官方）	讓 Claude 做的網頁更好看、更有設計感
skill-creator（官方）	幫你寫 Skill 的 Skill
i-have-adhd	讓回答更短，先給下一步行動

注意｜別人寫的 Skill 就是一份給 Claude 的指令，裝之前要看過內容、確認來源可信。

安裝別人的 Skill

1. **在 Code 分頁請 Claude 裝：貼上 GitHub 網址**
2. **或下載 .zip 上傳：「自訂」→ Skills → ＋ → Upload**
3. **裝完輸入 `/skills` 確認，再打開內容看一次**

示範：裝好 frontend-design 後，拿第 2 章的遊戲試

```
請用 frontend-design 這個 Skill，
重新設計我的貪食蛇的畫面，玩法不要改。
```

提示｜不點名也可以：說「重新設計畫面、更有設計感」，Claude 會比對 Skill 的說明自己決定要不要用；想直接指定，就輸入 `/frontend-design` 再接需求。

讓 Chat 也能用、分享給同學

1. **打包**
   請 Claude：「把 folder-organizer 打包成 .zip，存到這個資料夾」

2. **上傳**
   左側「自訂」→ Skills → ＋ → Create skill → Upload a skill

3. **分享**
   Skill 旁的「⋯」→ Share；或 Publish to org 發布給全校

> **提示**｜上傳後 Chat 和 Code 都能用（Code 輸入 `/skills` 會列在 claude.ai sync）。分享選項要學校管理員開啟。

RECAP

第 4 章回顧

- **判斷**：一句話講得清楚的事，不用存成 Skill
- **流程**：先做一次 → 喜歡 → 請 Claude 存成 Skill → 開新對話測試
- **資料夾整理**：先列計畫、確認後才動手、不刪原始檔
- 存完要**檢查**：名稱、描述、內容有沒有照你的意思
- 別人寫好的 Skill 可以裝，但要先看過再裝

CHAPTER 05

MCP：讓 Claude 長出手腳

用 Playwright 讓 Claude 自己打開網站、操作、截圖並回報問題

成品：測試截圖與問題回報

MCP

MCP 是什麼

- 一種讓 Claude 連到**外部工具**的標準接口
- 沒有 MCP，Claude 只能讀寫檔案、跑指令
- 裝上 Playwright 這個 MCP，它就多了一雙手：能**自己操作瀏覽器**

> **提示**｜你可以把 MCP 想成「給 Claude 的外接模組」。今天只裝一個。

MCP

Playwright MCP：讓 Claude 操作瀏覽器

打開網頁

自己開瀏覽器、輸入網址、調成手機尺寸

點擊與輸入

像真人一樣操作，從頭走完一個流程

截圖存檔

把畫面存成你指定檔名的圖片

找問題

發現按鈕沒反應、版面跑掉，並告訴你是哪一步

提示｜Playwright MCP 官方說明：https://github.com/microsoft/playwright-mcp

INSTALL

安裝 Playwright MCP（1）：打開設定檔

1. **打開設定**
   Claude Desktop → Settings → Developer

2. **按 Edit Config，用記事本打開**
   檔名 claude_desktop_config.json；裡面已經有內容的話，先不要刪

設定檔位置

```
Windows：%APPDATA%\Claude\claude_desktop_config.json
Mac：~/Library/Application Support/Claude/claude_desktop_config.json
```

提示｜Windows 可以把 %APPDATA%\Claude 貼到檔案總管的網址列，直接開啟資料夾。

INSTALL

安裝 Playwright MCP（2）：貼上設定

Windows：設定檔內容

```
{
  "mcpServers": {
    "playwright": {
      "command": "cmd",
      "args": ["/c", "npx", "@playwright/mcp@latest"]
    }
  }
}
```

注意｜Mac 先試：command 改 `npx`、args 改 `["@playwright/mcp@latest"]`；不行就把錯誤訊息貼給 Claude。已有其他設定時只加 `mcpServers` 那段，注意逗號。

INSTALL

安裝 Playwright MCP（3）：重啟並確認

1. **存檔**
   關掉記事本
2. **完全關閉 Claude**
   Windows：File → Exit，不是只關視窗
3. **重新開啟**
   回到 Settings → Developer，確認 playwright 在清單裡
4. **開新的 Code 工作階段**
   輸入 /mcp，跳出「MCP 伺服器」視窗，playwright 打勾就成功

> **提示**｜沒看到的話，通常是 JSON 的逗號或括號少了，把設定檔內容貼給 Claude 請它檢查。

INSTALL

終端機版怎麼裝 MCP

- Desktop 的設定檔是全域的，**每個專案都能用**
- 終端機版 Claude Code 用指令安裝；預設只在目前專案有效，加上 `--scope user` 才是每個專案都能用

終端機（二選一）

```
claude mcp add playwright npx @playwright/mcp@latest
claude mcp add --scope user playwright npx @playwright/mcp@latest
```

DEMO

示範：請 Claude 走一次完整流程並截圖

貼進對話區　聊天機器人版

```
重新啟動網頁伺服器。使用 iPhone 17 的尺寸（402 x 874）。
前往首頁，截圖並命名為 homepage.png。
輸入一個 PDF 裡有的問題並送出，確認回覆有附出處，
截圖並命名為 chat.png。
重新整理頁面，確認剛才的對話紀錄還在，
截圖並命名為 history.png。
三張截圖都存在目前的資料夾裡。
如果任何一步出現錯誤，把錯誤畫面也截圖下來，
並告訴我是哪一步發生的。
最後把瀏覽器關閉。
```

DEMO

如果你做的是貪食蛇

貼進對話區　貪食蛇版

重新啟動網頁伺服器。使用 1280 x 800 的視窗尺寸。
前往首頁，截圖並命名為 start.png。
用方向鍵讓蛇吃到至少一顆食物，確認分數有增加，
截圖並命名為 play.png。
故意讓蛇撞牆，確認出現遊戲結束畫面，截圖並命名為 ending.png。
三張截圖都存在目前的資料夾裡。
如果任何一步卡住或出錯，把畫面截下來，
並告訴我是哪一步發生的。
最後把瀏覽器關閉。

提示｜這段提示詞的結構可以套用到任何網站：尺寸、步驟、截圖檔名、出錯怎麼辦、收尾。

REVIEW

看它做了什麼：驗收

1　看截圖
打開資料夾，檔名對不對、畫面是不是真的長這樣

2　對照回報
Claude 說「成功」不算數，要跟截圖對得起來

3　看出錯那一步
有錯誤截圖的話，把它貼回去請 Claude 修

4　再跑一次
修完用同一段提示詞重測，確認問題真的消失

提示｜這和第 1 章一樣：AI 會說得很有自信，驗收還是你的工作。

RECAP

第 5 章回顧

- MCP 讓 Claude 能操作更多外部工具
- Playwright 適合：自動測試、截圖、檢查網頁
- 好的測試提示詞要寫清楚：**尺寸、步驟、截圖檔名、出錯怎麼辦、收尾**
- 用得越多，就越要注意**權限**：只裝你需要的

BONUS

把語音工具包成桌面 App

有剩餘時間再做：用 Electron 做一個按快捷鍵就能錄音、轉文字的小工具

Electron　快捷鍵　Whisper　剪貼簿

BONUS

網頁 vs 桌面 App

	網頁（第 2 章）	桌面 App
怎麼打開	瀏覽器輸入網址	電腦上的一個程式
字打在哪	只有自己的頁面	複製到剪貼簿，貼進任何軟體
快捷鍵	頁面被選中時才有用	在 Word、LINE 裡也能按
語音辨識	瀏覽器內建，免費	Whisper，用今天的 API key
分享	給網址就好	要打包成安裝檔（今天不做）

BONUS

Electron 是什麼

- 用寫網頁的方式（HTML、CSS、JavaScript）做桌面 App
- 像「自帶瀏覽器的殼」：把網頁包成一個獨立視窗
- VS Code、Discord、Notion 桌面版、Claude Desktop 都是用它做的
- 代價是檔案大（一兩百 MB），第一次要下載一陣子

> **提示**｜今天只用 `npm start` 在自己電腦上跑，不打包成安裝檔。

PROMPT

App 版 prompt

Plan Mode

在這個資料夾用 Electron 做一個桌面語音輸入小工具。
・快捷鍵 Ctrl+Shift+Space（Mac 用 Cmd+Shift+Space）：
  按一次開始錄音，再按一次停止
・錄好的聲音送到 OpenAI Whisper 轉成文字，輸出繁體中文
・轉好的文字自動複製到剪貼簿，並跳出系統通知「已複製」
・小視窗顯示狀態（待命／錄音中／轉換中）和最近 5 筆結果
・API key 放在 .env，不要寫進程式碼；你幫我建好 .env，我只貼上 key
・用 npm start 能執行就好，不用打包成安裝檔
先不要寫程式碼，給我計畫；做完告訴我怎麼啟動和測試。

提示｜開一個新資料夾，例如 voice-app。API key 用第 3 章那一把。

BONUS

測試與常見問題

症狀	可能原因／怎麼辦
npm install 很久	正在下載 Electron，等它跑完
按快捷鍵沒反應	可能被別的軟體占用，請 Claude 換一組
Mac 錄不到聲音	系統設定 → 隱私權與安全性 → 麥克風，允許終端機或 Claude
轉出來是簡體字	請 Claude 加上「轉成繁體中文」的處理
出現 401 錯誤	API key 沒放對，檢查 .env

提示｜怎麼測：打開 Word 或記事本，按快捷鍵講一句、再按一次，然後按 Ctrl+V 貼上。

REVIEW

今天你做了什麼

章節	作品	學到的方法
1 Chat 分頁	文件摘要、表格整理、營運儀表板	診斷、核算、驗收
2 靜態網站	貪食蛇、語音轉文字網頁（已部署）	Plan Mode、改需求、部署
3 動態網站	RAG 聊天機器人	RAG、後端、key、資料庫
4 Skill	你自己的 Skill	把好用的方法存起來
5 MCP	自動測試與截圖	擴充 Claude 的能力

TAKEAWAY

帶走的三件事

1　說清楚

用 PTCF 把需求說清楚

2　先看計畫

Plan Mode 降低做錯方向的成本

3　一定要驗收

AI 會錯，數字與事實要自己對

小結｜你不需要變成工程師，但你已經知道怎麼跟 AI 一起把點子做出來。

NEXT

接下來可以做什麼

- 把今天的作品換成**自己領域**的題材
- 把常用的做法再存成幾個 Skill
- 想做更完整的網站，回到 Plan Mode 一步一步來

WRAP-UP

收工前：請 Claude 關掉所有伺服器

貼到 Code 分頁

請幫我檢查今天在這台電腦上啟動、還在執行的東西：
・本機預覽和開發伺服器（例如 localhost:3000、5173、8000）
・聊天機器人的後端、語音桌面 App
・其他還在背景跑的 node 或 python 程式
先列給我看：名稱、port、在哪個資料夾，我確認後再關。
關完再檢查一次，告訴我還有沒有東西在跑。不要刪除任何檔案。

小結｜已經部署到 Vercel 的網站不受影響，網址還是能用。今天的 API key 課後會關閉，`.env` 不要上傳或分享。

Vibe Coding 跨領域實作工作坊

寫程式的門檻降低了，但沒有消失

你不用會寫，但你一定要看得懂驗收

今天做出來的每一個東西，都是從一句「我想要……」開始的。
差別只在於，你有沒有把那句話講清楚。

今天的問題，現在都可以問

謝謝大家！